

MediaPlayer - plik do nauki przed oddaniem projektu

1. Najkrótsze wyjaśnienie projektu

MediaPlayer to webowy odtwarzacz wideo napisany jako projekt studencki. Pozwala dodać kilka filmów do kolejki, odtwarzać je po kolei, dzielić każdy film na sekcje czasowe, dodawać komentarze do sekcji i pokazywać sekcje na osi czasu. Opis całej playlisty można zapisać do pliku JSON i później ponownie wczytać.

Player działa w dwóch wariantach:

1. jako samodzielna strona `index.html`, która dodatkowo ma formularze uploadu, dodania filmu po URL oraz importu/eksportu,
2. jako komponent do osadzenia na już istniejącej stronie, co pokazuje `embed-demo.html`.

Najważniejsza rzecz do zapamiętania: **logika samego playera jest w pliku `media-player.js`, a `app.js` obsługuje tylko stronę demonstracyjną**. Dzięki temu player można wykorzystać osobno.

2. Użyte technologie

Frontend

- **HTML5** - struktura strony oraz element `<video>` do odtwarzania filmu,
- **CSS3** - wygląd playera, kolejki, sekcji i układu strony,
- **JavaScript ES6** - cała logika playera, kolejki, sekcji, komentarzy, osi czasu oraz importu/eksportu.

Nie ma Reacta, Vue ani Angulara. Projekt jest celowo prosty i działa na zwykłym JavaScript.

Backend

- **Node.js** - środowisko uruchomieniowe serwera,
- **Express** - prosty serwer HTTP i udostępnianie katalogu `public`,
- **Multer** - odbieranie plików wideo wysyłanych z formularza.

FFmpeg - zewnętrzny program uruchamiany przez Node.js do konwersji AVI/MKV/DivX/Xvid i innych formatów do MP4 H.264/AAC.

Dane

- **JSON** - format importu i eksportu opisu playera,
 - **localStorage** - zapis aktualnej kolejki i sekcji w przeglądarce,
 - system plików - oryginalny upload trafia chwilowo do `temp-uploads`, a gotowy MP4 do `public/uploads`.
-

3. Struktura projektu i rola plików

```
MediaPlayer-project/
|-- server.js
|-- package.json
|-- temp-uploads/
|-- README.md
|-- CHECKLIST_ODDANIA.md
|-- public/
|   |-- index.html
```

MediaPlayer - materiał do nauki

```
| | -- embed-demo.html
| | -- assets/
| | | -- css/
| | | | -- player.css
| | | | -- app.css
| | | -- js/
| | | | -- media-player.js
| | | | -- app.js
| | -- data/
| | | -- sample-description.json
| | -- media/
| | | -- sample.mp4
| | -- uploads/
|-- docs/
```

server.js

Uruchamia Express, udostępnia pliki z katalogu public i ma endpoint POST /api/upload. Multer zapisuje plik do temp-uploads, a następnie server.js uruchamia FFmpeg przez `child_process.spawn()`. Film jest konwertowany do MP4 H.264/AAC, zapisywany w public/uploads, a oryginał tymczasowy jest usuwany.

public/index.html

Samodzielna strona projektu. Ma panel do:

- uploadu filmu,
- dodania filmu po adresie,
- importu JSON,
- eksportu JSON,
- wyczyszczenia danych lokalnych.

W środku strony jest element:

```
<div id="player"></div>
```

W tym miejscu JavaScript tworzy właściwy player.

public/assets/js/media-player.js

Najważniejszy plik projektu. Zawiera klasę `MediaPlayer`. Klasa buduje interfejs, przechowuje playlistę i obsługuje wszystkie funkcje związane z odtwarzaniem.

public/assets/js/app.js

Łączy samodzielną stronę z klasą `MediaPlayer`. Obsługuje formularze, wywołuje `fetch()` do uploadu i zapisuje dane do `localStorage`.

player.css

Style wielokrotnego użytku. Ten plik trzeba dołączyć również wtedy, gdy player jest osadzany na innej stronie.

app.css

Style tylko dla strony `index.html`. Nie są potrzebne do samego komponentu.

embed-demo.html

Pokazuje, że player może działać na obcej/istniejącej stronie. Dołącza tylko `player.css` i `media-player.js`, a potem tworzy obiekt przez `new MediaPlayer(...)`.

4. Jak działa klasa `MediaPlayer`

Player jest zrobiony jako klasa JavaScript:

```
class MediaPlayer {
  constructor(root, options = {}) {
    // ...
  }
}
```

`root` to element HTML, w którym ma pojawić się player. Może to być selektor tekstowy, np. `#player`, albo bezpośrednio element DOM.

Przykład:

```
const player = new MediaPlayer('#player', {
  playlist: [
    {
      title: 'Mój film',
      src: '/media/film.mp4',
      sections: []
    }
  ]
});
```

Najważniejsze pola obiektu

- `this.root` - element HTML komponentu,
 - `this.data` - dane playera, głównie `playlist`,
 - `this.currentIndex` - indeks aktualnie wybranego filmu,
 - `this.activeSectionIndex` - sekcja, w której aktualnie znajduje się czas filmu,
 - `this.video` - referencja do elementu `<video>`.
-

5. Model danych

Dane mają prostą strukturę:

```
{
  "version": 1,
  "playlist": [
    {
      "id": "media-...",
      "title": "Film demonstracyjny",
      "src": "/media/sample.mp4",
      "sections": [
        {
          "id": "section-...",
          "title": "Początek",

```

```

    "start": 0,
    "end": 4,
    "comments": [
      "Przykładowy komentarz"
    ]
  }
]
}

```

Czyli:

- player ma playlist,
- playlista ma filmy,
- film ma sections,
- sekcja ma czas start, end i tablicę comments.

id jest generowane po stronie JavaScript, głównie po to, żeby obiekty miały własne identyfikatory.

6. Kolejka odtwarzania

Film dodaje metoda `addMedia(media)`.

Metoda:

1. sprawdza, czy film ma `src`,
2. normalizuje dane,
3. dodaje film do tablicy `playlist`,
4. jeśli jest to pierwszy film, od razu go ładuje,
5. odświeża interfejs,
6. wysła zdarzenie informujące o zmianie danych.

Zmiana filmu odbywa się przez `loadMedia(index)`.

Przy końcu filmu wykonywane jest:

```

this.video.addEventListener('ended', () => {
  this.next();
});

```

Metody `next()` i `previous()` zmieniają indeks. Kolejka działa w pętli, czyli po ostatnim filmie następny jest pierwszy.

Film można również usunąć z kolejki przyciskiem `x`.

7. Sekcje filmu

Sekcja opisuje fragment filmu, np. od 20 do 50 sekundy.

Przykład:

```
{
  title: 'Omówienie kodu',
  start: 20,
  end: 50,
  comments: []
}
```

Metoda `addSection()` sprawdza m.in.:

- czy istnieje aktualny film,
- czy podano tytuł,
- czy czasy są liczbami,
- czy `end > start`,
- czy sekcja nie wykracza poza długość filmu.

Po dodaniu sekcje są sortowane po czasie początku.

Przyciski **Start = teraz** i **Koniec = teraz** wpisują do formularza aktualny czas filmu z `video.currentTime`.

8. Komentarze do sekcji

Każda sekcja ma tablicę `comments`.

Dodanie komentarza:

1. użytkownik wpisuje tekst,
2. `addCommentFromInput()` znajduje odpowiednią sekcję,
3. dopisuje komentarz do tablicy,
4. odświeża widok,
5. zapis danych jest uruchamiany przez zdarzenie `mediaplayer:change`.

Komentarz można też usunąć.

Nie ma autora komentarza ani daty, ponieważ projekt nie ma logowania i bazy danych. To jedno z celowych uproszczeń.

9. Oś czasu

Oś czasu to zwykły element HTML z pozycjonowanymi elementami reprezentującymi sekcje.

Po załadowaniu metadanych filmu przeglądarka zna jego długość w `video.duration`. Dla każdej sekcji obliczane są procenty:

```
pozycja od lewej = start / długość filmu * 100%
szerokość = (end - start) / długość filmu * 100%
```

Dzięki temu sekcje są narysowane proporcjonalnie do długości filmu.

Biały marker postępu jest przesuwany przy zdarzeniu `timeupdate`.

Kliknięcie osi czasu oblicza procent miejsca kliknięcia i ustawia:

```
video.currentTime = ratio * video.duration;
```

MediaPlayer - materiał do nauki

Kliknięcie bezpośrednio zaznaczonej sekcji przenosi do jej czasu start.

10. Import i eksport JSON

Eksport

`exportDescription()` zwraca obiekt z wersją formatu, datą eksportu i playlistą.

`downloadDescription()`:

1. wykonuje `JSON.stringify`,
2. tworzy Blob,
3. tworzy tymczasowy adres przez `URL.createObjectURL`,
4. programowo klika w link z atrybutem `download`.

Import

`loadDescription(input)` sprawdza, czy `playlist` jest tablicą, normalizuje dane i ustawia pierwszy film jako aktualny.

Na stronie `app.js` plik jest czytany przez:

```
const text = await file.text();
const data = JSON.parse(text);
player.loadDescription(data);
```

11. localStorage

`localStorage` to pamięć dostępna w przeglądarce. Projekt używa jej do prostego zapisu konfiguracji playera bez bazy danych.

`app.js` nasłuchuje zdarzenia:

```
playerRoot.addEventListener('mediaplayer:change', (event) => {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(event.detail));
});
```

Po ponownym otwarciu strony dane są odczytywane i przekazywane do konstruktora playera.

Ważne: `localStorage` przechowuje opis, a nie sam film. Po uploadzie gotowy, przekonwertowany MP4 jest plikiem na serwerze.

12. Jak działa upload filmu

Na stronie użytkownik wybiera plik. `app.js` tworzy `FormData` i wysyła plik do:

`POST /api/upload`

W `server.js` proces wygląda tak:

- Multer sprawdza rozszerzenie i zapisuje oryginał do `temp-uploads`,
- Node.js uruchamia FFmpeg przez `child_process.spawn()`,

MediaPlayer - materiał do nauki

- FFmpeg konwertuje wideo do H.264 i audio do AAC, a wynik zapisuje jako MP4 w public/uploads,
- oryginał tymczasowy jest usuwany, a backend zwraca JSON z adresem gotowego MP4.

Przykładowa odpowiedź:

```
{
  "title": "wyklad-1",
  "src": "/uploads/1720000000-wyklad-1.mp4",
  "size": 1234567,
  "originalFormat": ".avi",
  "outputFormat": ".mp4",
  "transcoded": true
}
```

Frontend bierze src i dodaje gotowy MP4 do kolejki przez player.addMedia(). Player nie musi wiedzieć, czy plik źródłowy był AVI, MKV czy MP4.

Warto rozróżniać kontener i kodek: MP4, MKV i AVI są kontenerami, a H.264, DivX i Xvid są kodekami wideo. Sama końcówka pliku nie gwarantuje zgodności z przeglądarką.

Najważniejsze opcje FFmpeg w projekcie to libx264 dla H.264, aac dla dźwięku, yuv420p dla zgodności oraz +faststart, żeby metadane MP4 były na początku pliku.

13. Dlaczego player można osadzić na innej stronie

Komponent jest oddzielony od strony aplikacji.

Do osadzenia potrzebne są tylko:

```
<link rel="stylesheet" href="/assets/css/player.css">
<div id="my-player"></div>
<script src="/assets/js/media-player.js"></script>
```

Następnie tworzy się obiekt:

```
new MediaPlayer('#my-player', {
  playlist: [
    { title: 'Film', src: '/video/film.mp4', sections: [] }
  ]
});
```

app.css i app.js nie są konieczne. To jest najważniejszy argument, gdy prowadzący zapyta, w jaki sposób spełniony jest wymóg wersji osadzanej.

14. Co dzieje się po kolei po uruchomieniu strony

1. Przeglądarka ładuje index.html.
2. Ładowane są player.css i app.css.
3. Ładowany jest media-player.js, który definiuje klasę MediaPlayer.
4. Ładowany jest app.js.
5. app.js sprawdza localStorage.
6. Tworzony jest obiekt new MediaPlayer(...).
7. Konstruktor tworzy HTML playera metodą renderLayout().

MediaPlayer - materiał do nauki

8. `bindEvents()` podłącza obsługę kliknięć i zdarzeń filmu.
 9. Ładowany jest pierwszy film.
 10. Po `loadedmetadata` budowana jest oś czasu.
 11. Przy każdej zmianie danych emitowane jest `mediaplayer:change`.
 12. `app.js` zapisuje nowy opis do `localStorage`.
-

15. Jakie podejścia/wzorce są tu użyte

Nie jest to projekt z rozbudowanymi wzorcami architektonicznymi, ale można wskazać kilka prostych pomysłów:

Komponent

`MediaPlayer` jest zamkniętym komponentem, który dostaje element DOM i dane wejściowe.

Rozdzielenie odpowiedzialności

- `media-player.js` - logika playera,
- `app.js` - logika strony,
- `server.js` - backend, upload i uruchamianie FFmpeg,
- CSS - wygląd,
- HTML - struktura strony.

Delegacja zdarzeń

Zamiast dodawać osobny listener do każdego dynamicznego przycisku, jeden listener jest podpięty do `this.root`, a przycisk rozpoznawany jest po `data-action`.

Zdarzenie własne

Komponent emituje `CustomEvent('mediaplayer:change')`. Dzięki temu sam komponent nie musi wiedzieć, czy dane są zapisywane do `localStorage`, bazy czy gdzieś indziej.

To przypomina prosty mechanizm obserwatora: komponent informuje o zmianie, a zewnętrzny kod reaguje.

16. Co jest celowo uproszczone

To ważne, bo prowadzący może zapytać, czego brakuje.

- brak logowania użytkowników,
- brak bazy danych,
- komentarze nie mają autora i daty,
- opisy są zapisywane lokalnie w przeglądarce albo w JSON,
- uploadowane pliki są trzymane na dysku serwera,
- brak transkodowania filmów do wielu jakości,
- brak napisów i miniatur,
- brak streamingu adaptacyjnego HLS/DASH,
- zewnętrzny URL filmu musi być dostępny dla przeglądarki,
- walidacja importowanego JSON jest podstawowa,
- interfejs jest prosty i nie ma rozbudowanej edycji sekcji.

Dobre zdanie na odpowiedź: **projekt rozwiązuje wymagany problem, ale nie próbuje być pełnym odpowiednikiem YouTube.**

17. Co można by rozwinąć w kolejnej wersji

- SQLite/PostgreSQL do przechowywania danych,
 - konta użytkowników,
 - osobna tabela komentarzy,
 - edycja sekcji bez usuwania,
 - przeciąganie kolejności filmów,
 - generowanie miniatur,
 - HLS i kilka jakości filmu,
 - autoryzacja uploadu,
 - przechowywanie plików np. w S3,
 - API REST do zapisu playlist po stronie serwera,
 - testy automatyczne,
 - lepsza obsługa błędów i większa walidacja JSON.
-

18. Najważniejsze metody do zapamiętania

Metoda	Co robi
<code>constructor()</code>	tworzy obiekt i inicjalizuje player
<code>renderLayout()</code>	tworzy podstawowy HTML komponentu
<code>bindEvents()</code>	podłącza obsługę zdarzeń
<code>addMedia()</code>	dodaje film do kolejki
<code>loadMedia()</code>	ładuje wybrany film do <code><video></code>
<code>next() / previous()</code>	zmienia film w kolejce
<code>removeMedia()</code>	usuwa film z kolejki
<code>addSection()</code>	dodaje sekcję czasową
<code>deleteSection()</code>	usuwa sekcję
<code>addCommentFromInput()</code>	dodaje komentarz
<code>renderTimeline()</code>	rysuje sekcje na osi czasu
<code>updateProgressMarker()</code>	przesuwa wskaźnik bieżącego czasu
<code>exportDescription()</code>	zwraca aktualne dane w formie obiektu
<code>downloadDescription()</code>	pobiera dane jako JSON
<code>loadDescription()</code>	importuje dane
<code>emitChange()</code>	wysyła własne zdarzenie o zmianie

19. Pytania, które może zadać prowadzący

1. Co jest głównym celem projektu?

Stworzenie webowego playera wideo z kolejką, sekcjami, komentarzami do sekcji, osią czasu i importem/eksporem opisu. Player ma działać samodzielnie i jako komponent osadzany na innej stronie.

2. Z czego korzystasz do odtwarzania filmu?

Z natywnego elementu HTML5 `<video>`. JavaScript steruje jego właściwościami, np. `src`, `currentTime` i `duration`.

3. Gdzie przechowywana jest playlista?

W obiekcie JavaScript w `this.data.playlist`. Na stronie standalone jej kopia jest zapisywana do `localStorage`.

4. Czy localStorage przechowuje plik wideo?

Nie. Przechowuje tylko opis, czyli adres filmu, tytuł, sekcje i komentarze.

5. Gdzie zapisuje się uploadowany film?

Oryginał najpierw trafia do temp-uploads. Po konwersji gotowy MP4 jest zapisany w public/uploads, a plik tymczasowy jest usuwany.

6. Do czego służy Multer?

Do obsługi uploadu plików przesyłanych przez formularz multipart/form-data. Multer sam nie konwertuje wideo - tylko odbiera plik i zapisuje go tymczasowo.

7. Po co jest Express?

Udostępnia statyczną stronę i tworzy endpointy, m.in. `/api/upload` oraz `/api/status`.

8. Dlaczego nie użyłeś Reacta?

Nie był potrzebny do wymaganego zakresu. Projekt jest mały, więc czysty JavaScript wystarcza i łatwiej pokazać działanie komponentu bez procesu budowania.

9. Jak player rozpoznaje koniec filmu?

Nasłuchuje zdarzenia `ended` elementu `<video>` i wywołuje `next()`.

10. Skąd znasz długość filmu?

Po zdarzeniu `loadedmetadata` można odczytać `video.duration`.

11. Jak wyliczana jest pozycja sekcji na osi czasu?

$\text{start} / \text{duration} * 100\%$. Szerokość to $(\text{end} - \text{start}) / \text{duration} * 100\%$.

12. Jak działa kliknięcie osi czasu?

Z pozycji kliknięcia liczę procent szerokości elementu i ustawiam `video.currentTime`.

13. Jak sprawdzasz aktywną sekcję?

Przy `timeupdate` szukam sekcji, dla której aktualny czas jest pomiędzy `start` i `end`.

14. W jakim formacie eksportujesz dane?

JSON.

15. Dlaczego JSON?

Jest prosty, czytelny, dobrze współpracuje z JavaScript i łatwo go zapisać oraz ponownie wczytać.

16. Jak chronisz HTML przed wpisanym tekstem komentarza?

Przed wstawieniem tekstu do `innerHTML` używana jest funkcja `escapeHtml()`, która zamienia znaki specjalne, np. `<` i `>`.

17. Co robi `data-action`?

Oznacza rodzaj akcji przycisku. Jeden wspólny listener sprawdza `data-action` i wykonuje odpowiednią metodę.

18. Co daje `CustomEvent`?

Komponent może poinformować kod zewnętrzny, że jego stan się zmienił. W tym projekcie `app.js` reaguje na to zapisem do `localStorage`.

19. Jak spełniasz wymaganie osadzenia na innej stronie?

`media-player.js` i `player.css` są niezależne od `index.html`. Wystarczy dodać kontener i utworzyć `new MediaPlayer(...)`. Pokazuje to `embed-demo.html`.

20. Czy film z dowolnego URL zadziała?

Nie zawsze. Serwer z filmem musi pozwalać przeglądarce pobrać/odtworzyć zasób i format musi być wspierany przez przeglądarkę.

21. Co się stanie po odświeżeniu strony?

Jeżeli dane były zapisane w `localStorage`, `app.js` je odczyta i odtworzy playlistę.

22. Co oznacza `preload="metadata"`?

Przeglądarka może pobrać metadane filmu, np. długość, bez pobierania od razu całego pliku.

23. Czy projekt ma bazę danych?

Nie. W tej wersji nie jest potrzebna. To świadome uproszczenie.

24. Co jest największym ograniczeniem tej wersji?

Brak trwałego zapisu opisów po stronie serwera i brak użytkowników. Dodatkowo transkodowanie odbywa się synchronicznie podczas jednego requestu, więc duże filmy mogą długo się przetwarzać.

25. Jak projekt obsługuje AVI, MKV i DivX/Xvid?

Nie próbuję odtwarzać ich bezpośrednio w HTML5 `<video>`. Przy uploadzie backend uruchamia FFmpeg i zamienia plik do MP4 z H.264/AAC. Player dostaje już kompatybilny plik wynikowy.

26. Czy MKV to kodek?

Nie. MKV to kontener. Może zawierać różne kodeki wideo i audio.

27. Czy DivX/Xvid to to samo co AVI?

Nie. DivX i Xvid to kodeki wideo, a AVI jest kontenerem. Często występują razem, dlatego potocznie te nazwy są mieszane.

28. Dlaczego format wynikowy to H.264/AAC w MP4?

Bo taki zestaw jest szeroko obsługiwany przez współczesne przeglądarki i element HTML5 <video>.

29. Dlaczego AVI/MKV nie działa po samym dwukliku index.html?

Tryb file:// nie ma backendu i nie może uruchomić programu FFmpeg z systemu. Rozszerzona zgodność wymaga npm start.

30. Jak uruchomić projekt?

Najpierw sprawdzam ffmpeg -version, potem npm install i npm start, następnie otwieram http://localhost:3000.

20. Krótkie demo na zaliczeniu - kolejność

Jeżeli masz 3-5 minut, pokaż to w takiej kolejności:

1. Sprawdź ffmpeg -version, potem uruchom npm start.
2. Otwórz stronę główną.
3. Pokaż film demonstracyjny i kolejkę.
4. Dodaj drugi film po URL albo uploadem.
5. Przejdź do wybranego momentu filmu.
6. Kliknij **Start = teraz**, przewiń film, kliknij **Koniec = teraz** i dodaj sekcję.
7. Dodaj komentarz do sekcji.
8. Kliknij sekcję na osi czasu.
9. Wyeksportuj JSON.
10. Wyczyść dane i zaimportuj JSON ponownie.
11. Otwórz embed-demo.html i powiedz, że to ten sam MediaPlayer, ale na innej stronie.

To pokazuje praktycznie wszystkie wymagania bez przypadkowego błądzenia po kodzie, czyli rzecz rzadsza na zaliczeniach, niż ludzkość chciałaby przyznać.

21. Co warto umieć pokazać w kodzie

Najlepiej mieć zapamiętane miejsca:

- media-player.js - konstruktor klasy,
- bindEvents() - eventy <video> i delegacja kliknięć,
- addSection() - walidacja sekcji,
- renderTimeline() - procentowe pozycjonowanie sekcji,
- downloadDescription() - eksport JSON,
- loadDescription() - import JSON,
- app.js - fetch('/api/upload'),
- server.js - Multer, endpoint uploadu i transcodeToMp4(); FFmpeg używa libx264, AAC, yuv420p i +faststart,
- embed-demo.html - new MediaPlayer(...).